TECHNICAL SPECIFICATION

One-Time Post-Quantum Ring Signature

Based on Batch Deniable Commitment.
Proof of Concept.

Author: Anton A. Sokolov (acmxddk@gmail.com)

October 2026

This work is licensed under CC BY 4.0

Abstract

This document defines the formal technical specification for a One-Time Post-Quantum Ring Signature (**OT-PQRS**) scheme. The primitive is inherently built upon the interaction of two main core protocols: the Partially Fixed Merkle Tree (**PFMT**) and the Batch Deniable Commitment Scheme (**BDCS**). The **PFMT** serves as an ad-hoc group identity binder, allowing a Prover to embed a personal secret leaf value into a balanced tree layout at an independent, uniformly random path without disrupting unassociated ring public keys. The **BDCS** operates in tandem as a high-density privacy vector engine, committing compressed topological representations of the internal path variables into an oblivious multi-proof structure. During signature generation, these systems tightly intertwine to produce a compact, 128-bit post-quantum secure (**NIST Level 1 Compliance**) linkable proof of group membership that ensures total zero-knowledge signer anonymity and deterministic double-signing prevention under the Random Oracle Model.

Contents

1 Mathematical Base & Objectives **2**
1.1 Structural Properties and Security Goals 2

2 Partially Fixed Merkle Tree (PFMT) Protocol Mechanics **2**
2.1 Nomenclature Rationale 2
2.2 Initialization Phase 3
2.3 Secret Incorporation Phase 3
2.4 Population and Hierarchical Folding Phase 4

3 Cryptographic Key Invariants and Structuring **5**
3.1 Private Key Generation Layout (SK) 5
3.2 Public Key Generation Layout (PK) 5

4 Message Signing and Proof Generation **6**
4.1 Ring Construction and Subtraction 6
4.2 Signature Composition Sequence 6

5 Independent Signature Verification Pipeline **7**
5.1 Phase 1: BDC Binding Validation 7
5.2 Phase 2: PFMT Boundary Alignment Validation 8

6 Appendix: Security Analysis and Formal Proofs **8**
6.1 Anonymity (Signer Unlinkability) 8
6.2 Binding and Linkability (Double-Signing Prevention) 10
6.3 Unforgeability (EUF-CMA Resistance) 10

7 Appendix: Architectural Flow and Structural Diagrams **12**
7.1 Hierarchical Key Isolation and Blinding Pipeline 12
7.2 Phase-Split Key Generation Strategy 13
7.3 The Signature Composition Loop 13
7.4 PFMT Spatial Slicing and Layerwise Merkle Tree Folding 14

1 Mathematical Base & Objectives

The specified primitive defines a One-Time Post-Quantum Ring Signature (**OT-PQRS**) framework. It reduces the computational problem of proving a signer's identity membership within an ad-hoc group of public keys down to a localized verification instance of the Batch Deniable Commitment Scheme (**BDCS**). Linkability is structurally implied by the one-time enforcement invariant of the protocol.

1.1 Structural Properties and Security Goals

- **Anonymity Ceiling ($\geq$ 128-bit PQ):** A completed signature exposes zero identifying traits or tracing properties pointing to the active signer within a ring of $\mathbf{2^V - 1}$ unassociated public keys. Anonymity is bound strictly to the zero-knowledge deniability parameters of the underlying **BDCS** layers.
- **Linkability Constraint (Double-Signing Enforcement):** The scheme is strictly one-time per public key instance. If a private key holder issues more than one signature on behalf of their public key, a deterministic linking tag **I** collision occurs. This structural intersection uniquely extracts the underlying secret parameters, revealing the signer's identity and preventing identity fraud.
- **Post-Quantum Unforgeability:** Forgery resistance is bound tightly to the collision and preimage resistance of `sha512` and `sha256` primitives, securing full compliance up to **NIST Level 1** post-quantum thresholds ($\mathbf{2^{128}}$ quantum operations).
- **Algebraic Cleanliness:** Every spatial vector translation step, delta manipulation, and node accumulation layer utilizes pointwise **bitwise XOR** ($\oplus$) operations over flat 512-bit binary blocks. This eliminates arithmetic overflow states, drop-carries, and algebraic field leakage profiles.

2 Partially Fixed Merkle Tree (PFMT) Protocol Mechanics

The Partially Fixed Merkle Tree (**PFMT**) manages the deterministic expansion of private seeds, calculating localized path deltas that structurally tie the signer's identity scalar into an ad-hoc public key layer.

2.1 Nomenclature Rationale

The cryptographic architecture is designated as a *Partially Fixed* Merkle Tree due to its distinct, three-stage progressive state mutation constraints:

- **Root Fixation (Initialization Phase):** The master root boundary hash **Rt** is unconditionally fixed from pseudorandom seed expansions prior to any secret data integration, establishing an unalterable upper topological limit.
- **Path Fixation (Secret Incorporation Phase):** The localized hash inputs along the isolated target secret trajectory are permanently bound and frozen when the secret identity scalar **TSI** is injected, forming immutable internal delta trajectories.
- **Global Tree Fixation (Population Phase):** The remaining structural dimensions of the balanced tree are globally stabilized when the ad-hoc public keys are allocated across the open leaves layer, finalizing the complete topological matrix for verification.

2.2 Initialization Phase

Given a 512-bit private random seed **TRS**, the initialization module expands a sequence of **V** pairs of 512-bit scalars denoted by the collection $\{(\mathbf{EL_i}, \mathbf{ER_i})\}$, indexed levelwise from the leaves up to the root $\mathbf{i} = \mathbf{0} \ldots \mathbf{V-1}$:

1. For each height tier index **i** from 0 up to $\mathbf{V-1}$, evaluate parallel domain-separated string states:

$$\mathbf{EL_i} = \texttt{sha512}(\mathbf{TRS} \parallel \text{"L"} \parallel \mathbf{i})$$
$$\mathbf{ER_i} = \texttt{sha512}(\mathbf{TRS} \parallel \text{"R"} \parallel \mathbf{i})$$

2. Compute the fixed binary tree root parameter **Rt** directly from the final top-tier layer:

$$\mathbf{Rt} = \texttt{sha512}(\mathbf{EL_{V-1}} \parallel \mathbf{ER_{V-1}})$$

3. Sample from **TRS** an independent, uniformly random $\mathbf{V-1}$ bitpath routing vector denoted as:

$$\mathbf{TBP} = \{\mathbf{B_1}, \mathbf{B_2}, \ldots, \mathbf{B_{V-1}}\}, \text{ where } \mathbf{B_i} = \texttt{sha512}(\mathbf{TRS} \parallel \text{"Bitpath"} \parallel \mathbf{i})$$

As an alternative, this bitpath vector **TBP** can also be taken from the private input rather than generated from the tree seed **TRS**.

4. Initialize two private 512-bit scalar vectors denoted as Secret Left (**SL**) and Secret Right (**SR**) of size $\mathbf{V-1}$. For index height levels $\mathbf{i} = \mathbf{1} \ldots \mathbf{V-1}$, let the target delta value be calculated as:

$$\mathbf{\Delta_i} = \mathbf{E}_{\text{chosen},\mathbf{i}} \oplus \texttt{sha512}(\mathbf{EL_{i-1}} \parallel \mathbf{ER_{i-1}})$$

where $\mathbf{E}_{\text{chosen},\mathbf{i}} = \mathbf{EL_i}$ if $\mathbf{B_i} = \mathbf{0}$, and $\mathbf{ER_i}$ if $\mathbf{B_i} = \mathbf{1}$. This delta is distributed into the private vectors matching the **TBP** routing path coordinates:

$$\text{If } \mathbf{B_i} = \mathbf{0}: \quad \mathbf{SL_i} = \mathbf{\Delta_i}, \quad \mathbf{SR_i} = \text{NaN}$$
$$\text{If } \mathbf{B_i} = \mathbf{1}: \quad \mathbf{SR_i} = \mathbf{\Delta_i}, \quad \mathbf{SL_i} = \text{NaN}$$

2.3 Secret Incorporation Phase

Given a 512-bit personal secret identity scalar **TSI**, the Prover samples an independent random coin-flip bit denoted as $\mathbf{B_0} \in \{\mathbf{0}, \mathbf{1}\}$.

1. Prepend $\mathbf{B_0}$ into the absolute front of the routing vector **TBP**, expanding its dimensions from size $\mathbf{V-1}$ to exactly size **V**.

2. Compute the initial base entry level secret delta variable $\mathbf{\Delta_0}$:

$$\mathbf{\Delta_0} = \mathbf{E}_{\text{chosen},\mathbf{0}} \oplus \mathbf{TSI}$$

where $\mathbf{E}_{\text{chosen},\mathbf{0}} = \mathbf{EL_0}$ if $\mathbf{B_0} = \mathbf{0}$, and $\mathbf{ER_0}$ if $\mathbf{B_0} = \mathbf{1}$.

3. Prepend this secret delta entry into the front of the sparse vectors tracking space at index position $\mathbf{0}$:

$$\begin{aligned} \text{If } \mathbf{B_0} = \mathbf{0}: &\quad \mathbf{SL_0} = \mathbf{\Delta_0}, \quad \mathbf{SR_0} = \text{NaN} \\ \text{If } \mathbf{B_0} = \mathbf{1}: &\quad \mathbf{SR_0} = \mathbf{\Delta_0}, \quad \mathbf{SL_0} = \text{NaN} \end{aligned}$$

The remaining non-assigned companion slots across all height levels $\mathbf{i} = \mathbf{0} \ldots \mathbf{V} - \mathbf{1}$ temporarily preserve their **NaN** values until the tree is populated.

2.4 Population and Hierarchical Folding Phase

Given an array of $\mathbf{2^V}$ 512-bit scalars representing the input leaves layer, the tree folding process requires the secret leaf **TSI** to be located at the unique position determined by **TBP** within this layer, while the remaining $\mathbf{2^V - 1}$ slots can be occupied by arbitrary scalars.

The tree folds bottom-up to the root **Rt** levelwise. At level $\mathbf{i} = \mathbf{0} \ldots \mathbf{V} - \mathbf{1}$:

1. Isolate the target sibling component node sitting immediately opposite the secret path: $\mathbf{C_Node} = \text{Node}[\mathbf{TBP_i} \oplus \mathbf{1}]$.

2. Compute the companion delta scalar element:

$$\mathbf{\Delta_{C,i}} = \mathbf{E}_{\text{companion},\mathbf{i}} \oplus \mathbf{C_Node}$$

where $\mathbf{E}_{\text{companion},\mathbf{i}} = \mathbf{ER_i}$ if $\mathbf{TBP_i} = \mathbf{0}$, and $\mathbf{EL_i}$ if $\mathbf{TBP_i} = \mathbf{1}$.

3. Distribute this companion delta into the vacant **NaN** placeholder positions of the private secret vectors:

$$\begin{aligned} \text{If } \mathbf{TBP_i} = \mathbf{0}: &\quad \mathbf{SR_i} = \mathbf{\Delta_{C,i}} \\ \text{If } \mathbf{TBP_i} = \mathbf{1}: &\quad \mathbf{SL_i} = \mathbf{\Delta_{C,i}} \end{aligned}$$

4. Compress the current layer size down by half. For every sequential node pair index $\mathbf{j}$, compute the parents for the next level:

$$\text{ParentNode}_\mathbf{j} = \texttt{sha512}\big((\text{Node}_{\mathbf{2j}} \oplus \mathbf{SL_i}) \parallel (\text{Node}_{\mathbf{2j+1}} \oplus \mathbf{SR_i})\big)$$

5. Repeat this process recursively for each level up to $\mathbf{V} - \mathbf{1}$ until the root state **Rt** is reconstructed.

3 Cryptographic Key Invariants and Structuring

The cryptographic lifecycle translates raw asymmetric keys into a dual-linked commitment format, creating the unforgeable binding layer before public broadcast. This dual-linkage structurally anchors two independent protocol vectors into a single cryptographic primitive:

- **The Identity Link:** Binds the public key to the signer's identity parameters through the master root **PFMT.Rt**.
- **The Blinding Link:** Locks the secret path-blinding components (**SL**, **SR**) directly to the random oracle parameters committed within **BDC.MTR**.

The final privately held commitment token **CMT** is computed by synthesizing both roots:

$$\mathbf{CMT} = \texttt{sha512}(\text{PFMT.}\mathbf{Rt} \parallel \text{BDC.}\mathbf{MTR})$$

3.1 Private Key Generation Layout (SK)

A Prover's private signature credential consists of three independent security components:

$$\mathbf{SK} = (\mathbf{SKP}, \mathbf{TRS}, \mathbf{GGMR})$$

- **SKP**: A 512-bit secret signing identity scalar.
- **TRS**: A 512-bit random tracking root configuration seed.
- **GGMR**: A 256-bit random master seed for GGM tree mask derivation.

3.2 Public Key Generation Layout (PK)

To compile a 512-bit unlinked Public Key, the Prover executes the following protocol prior to ring assignment:

1. Initialize an independent instance of the PFMT engine using the private seed: PFMT.$\mathbf{TRS} = \mathbf{TRS}$, and extract the 512-bit tree root PFMT.**Rt**.
2. Incorporate the personal secret identity scalar **SKP** as the secret leaf entry $\mathbf{TSI} = \mathbf{SKP}$, evaluating the tracking vectors **SL** and **SR** of size **V**. At this initialization juncture, each height level $\mathbf{i} = \mathbf{0} \ldots \mathbf{V-1}$ contains exactly one valid delta entry $\mathbf{\Delta_i}$ written to one of **SL** and **SR**, depending of $\mathbf{TBP_i}$, while the alternative companion remains an empty NaN placeholder.
3. Extract the **V** valid, populated delta scalars from across the vectors **SL** and **SR**, and process each independently through a standard `sha256` circuit to generate a compressed 256-bit path hash vector **HCPS** of size **V**:

   $$\mathbf{HCPS_i} = \texttt{sha256}(\mathbf{\Delta_i}) \quad \text{for } \mathbf{i} = \mathbf{0} \ldots \mathbf{V-1}$$

4. Initialize a Batch Deniable Commitment (**BDC**) engine instance using **GGMR** as the master root seed. Commit the path hash vector **HCPS** to the **BDC**, generating the corresponding 256-bit BDC Merkle Tree Root (BDC.**MTR**).
5. Compute the privately held linking token commitment value **CMT**:

$$\mathbf{CMT} = \texttt{sha512}(\text{PFMT.}\mathbf{Rt} \parallel \text{BDC.}\mathbf{MTR})$$

6. Output the final public key **PK** by masking the private scalar behind the linear XOR transformation:

$$\mathbf{PK} = \mathbf{SKP} \oplus \mathbf{CMT}$$

4 Message Signing and Proof Generation

Given an ad-hoc public verification ring $\mathbf{RNG_{in}}$ containing exactly $\mathbf{2^V - 1}$ 512-bit public keys along with a 512-bit message token **m**, a Prover holding valid private key credentials **SK** matching a constituent public key **PK** within the ring generates an anonymous proof signature.

4.1 Ring Construction and Subtraction

The signature verification routine operates over an expanded group array **RNG** containing exactly $\mathbf{2^V}$ 512-bit entries. Prior to message binding expansion, the input ring contains $\mathbf{2^V - 1}$ public keys, within which the signing identity public key **PK** is located exactly at the index position determined by the binary evaluation of the bitpath vector **TBP** generated during the public key creation stage. The remaining $\mathbf{2^V - 2}$ slots within the original ring are occupied by arbitrary public scalars.

To bind the transaction context, the Prover extracts the unique linking tag indicator value **I** straight from the internal credential parameters cache:

$$\mathbf{I} = \mathbf{CMT} = \texttt{sha512}(\text{PFMT.}\mathbf{Rt} \parallel \text{BDC.}\mathbf{MTR})$$

The Prover subsequently computes an independent cryptographic message anchor **M**:

$$\mathbf{M} = \texttt{sha512}(\mathbf{m} \parallel \mathbf{I}) \oplus \mathbf{I}$$

The Prover appends **M** directly to the absolute tail of the ring at the final index slot ($\mathbf{2^V - 1}$), expanding the active ring size to its balanced threshold of $\mathbf{2^V}$ elements. The target leaf spectrum layer **Leaf** is subsequently extracted by executing a pointwise bitwise XOR subtraction pass using the linking tag **I** element-wise across the compiled structure:

$$\mathbf{Leaf_k} = \mathbf{RNG_k} \oplus \mathbf{I} \quad \text{for } \mathbf{k} = \mathbf{0} \ldots \mathbf{2^V - 1}$$

This processed leaf spectrum vector is passed straight as the uniform input base layer into the PFMT hierarchical folding phase.

4.2 Signature Composition Sequence

Upon completion of the leaf level preparations, the Prover finalizes the proof parameters through the following deterministic execution sequence:

1. Pass the extracted leaf spectrum layer through the **PFMT** population and folding engine. This process completely populates the remaining vacant companion placeholder positions inside the tracking vectors **SL** and **SR** across all height levels $\mathbf{i} = \mathbf{0} \ldots \mathbf{V - 1}$.

2. Compose a 512-bit vector pair structure denoted by **SLR** containing exactly **V** pairs, derived directly from the now fully populated private vectors:

$$\mathbf{SLR_i} = (\mathbf{SL_i}, \mathbf{SR_i}) \quad \text{for } \mathbf{i} = \mathbf{0} \ldots \mathbf{V-1}$$

3. Note that since $\mathbf{SL_i}$ and $\mathbf{SR_i}$ are natively bound to the left and right positions of the level-wise tree folding structure respectively, no secondary path-dependent reordering or auxiliary array mapping is required, naturally shielding the signer's identity.

4. Compress the complete presentation pairs into a $\mathbf{2} \times \mathbf{V}$ 256-bit representation matrix by processing each element through a `sha256` hashing pass to construct the hashed signature tracking vector **HSLR**:

$$\mathbf{HSLR_i} = \big(\texttt{sha256}(\mathbf{SLR_{i,\text{left}}}),\ \texttt{sha256}(\mathbf{SLR_{i,\text{right}}})\big) = \big(\texttt{sha256}(\mathbf{SL_i}),\ \texttt{sha256}(\mathbf{SR_i})\big)$$

5. Invoke the **BDC** proof routine over the **HSLR** array to generate the tracking block structure **BDCP** = BDC.**Proof**, proving that the constituent path elements correspond to the public commitment anchor BDC.**MTR**.

6. Output the complete signature packet **Sig**:

$$\mathbf{Sig} = (\mathbf{RT} = \text{PFMT.}\mathbf{Rt},\ \mathbf{MTR} = \text{BDC.}\mathbf{MTR},\ \mathbf{SLR},\ \mathbf{BDCP} = \text{BDC.}\mathbf{Proof})$$

5 Independent Signature Verification Pipeline

A Verifier holding only the public ring $\mathbf{RNG_{in}}$ containing exactly $\mathbf{2^{V-1}}$ 512-bit entry keys, the 512-bit message token **m**, and the transmitted signature packet **Sig** validates the proof through a strict two-tier procedure.

5.1 Phase 1: BDC Binding Validation

1. Reject immediately if the size of the received **Sig**.**SLR** array of pairs does not equal **V**.

2. Map the 512-bit presentation signature pairs down to a 256-bit verification mapping vector by hashing each component element-wise:

$$\mathbf{HSLR_i} = \big(\texttt{sha256}(\mathbf{Sig.SLR_{i,\text{left}}}),\ \texttt{sha256}(\mathbf{Sig.SLR_{i,\text{right}}})\big) \quad \text{for } \mathbf{i} = \mathbf{0} \ldots \mathbf{V-1}$$

3. Forward the compiled vector **HSLR**, the structural path payload **Sig**.**BDCP**, and the public commitment tracking root **Sig**.**MTR** directly into the core BDC verification routine:

$$\text{Check}_{\text{BDC}} = \text{BDC.verifyProof}(\mathbf{Sig.MTR}, \mathbf{HSLR}, \mathbf{Sig.BDCP})$$

4. If $\text{Check}_{\text{BDC}}$ evaluates to `False`, terminate the pipeline immediately, reject the signature, and output a fault state.

5.2 Phase 2: PFMT Boundary Alignment Validation

1. Reconstruct the active public linking tag instance value $\mathbf{I}$ using the transmitted root tokens:

$$\mathbf{I} = \texttt{sha512}(\mathbf{Sig.RT} \parallel \mathbf{Sig.MTR})$$

2. Reconstruct the unaligned cryptographic message anchor position value $\mathbf{M}$ using the input context data:

$$\mathbf{M} = \texttt{sha512}(\mathbf{m} \parallel \mathbf{I}) \oplus \mathbf{I}$$

3. Append $\mathbf{M}$ directly to the absolute tail of the received public key verification ring to assemble a temporary validation structure $\mathbf{RNG}_{\text{vld}}$ containing exactly $\mathbf{2^V}$ elements.

4. Execute a pointwise bitwise XOR subtraction pass using the reconstructed linking tag $\mathbf{I}$ element-wise across $\mathbf{RNG}_{\text{vld}}$ to extract the target verification leaves layer:

$$\mathbf{Leaf_k} = \mathbf{RNG}_{\text{vld,k}} \oplus \mathbf{I} \quad \text{for } \mathbf{k} = \mathbf{0} \dots \mathbf{2^V - 1}$$

5. Progress up-tree layerwise from $\mathbf{i = 0}$ up to $\mathbf{V - 1}$. At each height tier, execute a parallel Merkle tree folding step by pairwise evaluating the adjacent node hashes combined with the broadcasted signature pairs:

$$\text{ParentNode}_{\mathbf{j}} = \texttt{sha512}\big((\text{Node}_{\mathbf{2j}} \oplus \mathbf{Sig.SLR}_{\mathbf{i},\text{left}}) \parallel (\text{Node}_{\mathbf{2j+1}} \oplus \mathbf{Sig.SLR}_{\mathbf{i},\text{right}})\big)$$

6. After completing the final top-tier compression pass, assert that the calculated root matches the transmitted reference proof value exactly:

$$\text{ParentNode}_{\mathbf{0}} == \mathbf{Sig.RT}$$

7. If this structural equality holds true, terminate the pipeline and emit an unconditional acceptance signal. If any equality verification step fails, reject immediately.

6 Appendix: Security Analysis and Formal Proofs

This section presents the formal post-quantum security proofs for the **OT-PQRS** primitive under the Random Oracle Model (**ROM**). All cryptographic hash functions, specifically `sha512` and `sha256`, are modeled as ideal random oracles capturing standard quantum preimage and collision resistance complexities.

6.1 Anonymity (Signer Unlinkability)

Theorem 1. *Assuming `sha512` and `sha256` act as ideal random oracles, no post-quantum adversary $\mathcal{A}$ can identify the true signer's public key $\mathbf{PK_t}$ within an ad-hoc ring $\mathbf{RNG}$ of size $\mathbf{2^{V-1}}$ with an advantage significantly greater than random guessing, even if $\mathcal{A}$ possesses all active signature payload blocks and the complete set of non-target private keys.*

Proof. Let $\mathcal{A}$ be a post-quantum adversary attempting to break signer anonymity. We define a standard indistinguishability game where the challenger selects a uniformly random bit $\mathbf{b} \in \{\mathbf{0}, \ldots, \mathbf{2^{V-1}-1}\}$ corresponding to the target index of the legitimate signer's public key $\mathbf{PK_b}$ inside the verification ring **RNG**. The challenger generates a valid signature $\mathbf{Sig} = (\mathbf{RT}, \mathbf{MTR}, \mathbf{SLR}, \mathbf{BDCP})$ on behalf of $\mathbf{PK_b}$ for a message $\mathbf{m}$.

1. **Statistical Uniformity of the Vector Layer:** The transmitted vector **Sig**.**SLR** contains **V** pairs of 512-bit scalars extracted from **SL** and **SR**. Because the operations are pointwise XOR ($\oplus$), for every level $\mathbf{i}$, the elements satisfy:

$$\begin{aligned} \text{Node}_{\mathbf{2j}} \oplus \mathbf{Sig.SLR_{i,\text{left}}} &= \mathbf{EL_i} \\ \text{Node}_{\mathbf{2j+1}} \oplus \mathbf{Sig.SLR_{i,\text{right}}} &= \mathbf{ER_i} \end{aligned}$$

By definition of the initialization phase, the true trajectory values are independently evaluated via domain-separated strings:

$$\begin{aligned} \mathbf{EL_i} &= \texttt{sha512}(\mathbf{TRS} \parallel \text{"L"} \parallel \mathbf{i}) \\ \mathbf{ER_i} &= \texttt{sha512}(\mathbf{TRS} \parallel \text{"R"} \parallel \mathbf{i}) \end{aligned}$$

Since **TRS** is a secure random scalar held privately by the signer, the outputs $\mathbf{EL_i}$ and $\mathbf{ER_i}$ are distributed uniformly and independently across $\{0,1\}^{512}$ under the Random Oracle Model.

2. **Blinding Vector Mask Independence:** During the population phase, the Prover dynamically evaluates a companion delta $\mathbf{\Delta_{C,i}}$ for the unassigned node sitting opposite the path index. Because the operations are pointwise bitwise XOR ($\oplus$), the companion delta values inherit the perfect uniform distribution of the underlying random oracle outputs. The resulting tuple $\mathbf{Sig.SLR_i} = (\mathbf{SL_i}, \mathbf{SR_i})$ consists of one primary trajectory delta and one companion blinding delta arranged in strict left/right physical tree orientation. No position-dependent reordering occurs over **SL** and **SR**; hence, looking at the ordered presentation **Sig**.**SLR** provides zero statistical leakage regarding which component represents the authentic path coordinate.

3. **Zero-Knowledge Commitments Isolation:** The hashed representation vector **HSLR** is derived element-wise from **Sig**.**SLR**. This tracking structure is verified directly against the reference root **Sig**.**MTR** using the proof payload **Sig**.**BDCP**. The underlying **BDCS** framework satisfies absolute zero-knowledge deniability up to the 128-bit security floor. Prior to public broadcast, the public linking token is masked by the one-way commitment rule:

$$\mathbf{CMT} = \texttt{sha512}(\text{PFMT.}\mathbf{Rt} \parallel \text{BDC.}\mathbf{MTR})$$

This shields the roots behind an unbreakable preimage barrier. Therefore, the information view available to any quantum adversary $\mathcal{A}$ is statistically decoupled from the secret bitpath **TBP** or the target bit **b**. The probability of an optimal guess bounds exactly to:

$$\Pr[\mathcal{A} \text{ wins}] = \frac{\mathbf{1}}{\mathbf{2^{V-1}}} + \text{negl}(\mathbf{2^{128}})$$

This satisfies absolute zero-knowledge signer anonymity up to a full 128-bit post-quantum floor. □

6.2 Binding and Linkability (Double-Signing Prevention)

Theorem 2. *If a malicious signer attempts to issue two distinct signatures* $\mathbf{Sig_1}$ *and* $\mathbf{Sig_2}$ *for two different messages* $\mathbf{m_1} \neq \mathbf{m_2}$ *using the same private key credentials, the signatures will either produce an identical linking tag* $\mathbf{I_1} = \mathbf{I_2}$*, exposing the double-signing violation to public verifiers, or violate the collision resistance of* `sha512` *at a 128-bit post-quantum security threshold.*

Proof. Let a malicious adversary attempt to double-sign two distinct messages $\mathbf{m_1} \neq \mathbf{m_2}$ while avoiding detection by forcing alternative linking tags $\mathbf{I_1} \neq \mathbf{I_2}$. The linking tag parameter is deterministically bound to the public roots via the one-way mapping $\mathbf{I} = \texttt{sha512}(\mathbf{RT} \parallel \mathbf{MTR})$.

1. **Case A: The Adversary preserves the commitment roots ($\mathbf{RT_1} = \mathbf{RT_2}$ and $\mathbf{MTR_1} = \mathbf{MTR_2}$).** If the private commitments are preserved across both signatures, then by definition of the random oracle properties:

$$\mathbf{I_1} = \texttt{sha512}(\mathbf{RT_1} \parallel \mathbf{MTR_1}) = \texttt{sha512}(\mathbf{RT_2} \parallel \mathbf{MTR_2}) = \mathbf{I_2}$$

This yields a strict collision $\mathbf{I_1} = \mathbf{I_2}$ across the ledger. Because the linking tags intersect, the calculated message anchors split over the distinct messages:

$$\mathbf{M_1} = \texttt{sha512}(\mathbf{m_1} \parallel \mathbf{I}) \oplus \mathbf{I}$$
$$\mathbf{M_2} = \texttt{sha512}(\mathbf{m_2} \parallel \mathbf{I}) \oplus \mathbf{I}$$

Since $\mathbf{m_1} \neq \mathbf{m_2}$, then $\mathbf{M_1} \neq \mathbf{M_2}$ with overwhelming probability. The two evaluation trees now contain different node values at the final index leaf slot $(\mathbf{2^V - 1})$. Because the Merkle tree folding rules are entirely deterministic, the verifier's pipeline will process two disparate leaf layers. To force both asymmetric structures to collapse exactly to the identical fixed reference root $\mathbf{RT_1}$, the adversary is mathematically compelled to provide conflicting, contradictory scalar pairs within $\mathbf{Sig.SLR_1}$ and $\mathbf{Sig.SLR_2}$ for at least one height tier $\mathbf{i}$.

2. **The Single-Element Path Commitment Invariant:** During public key generation, the signer is restricted by step 3 of the protocol to commit only the single-element path hash vector $\mathbf{HCPS_i} = \texttt{sha256}(\mathbf{\Delta_i})$ straight into $\mathbf{BDC.MTR}$. Because the opposing companion slot is unpopulated (NaN) at keygen, it is completely absent from the master root commitment. When the adversary emits a conflicting pair to mask the message branch mutation, the verified path checks against $\mathbf{BDC.MTR}$ will instantly fail unless the adversary breaks the preimage resistance of `sha256`. The conflicting opening reveals the secret vector orientation $\mathbf{TBP}$, stripping the adversary of anonymity.

3. **Case B: The Adversary attempts to manipulate the roots ($\mathbf{RT_1} \neq \mathbf{RT_2}$ or $\mathbf{MTR_1} \neq \mathbf{MTR_2}$) under a single Public Key.** The public key is structurally bound via the XOR equation $\mathbf{PK} = \mathbf{SKP} \oplus \mathbf{CMT}$. If the adversary attempts to forge alternative roots to yield $\mathbf{I_1} \neq \mathbf{I_2}$, they must find an alternative commitment token $\mathbf{CMT'}$ that maps to a valid private signing key $\mathbf{SKP'}$. This requires finding a global collision across the synthesized tree paths. By applying Grover's quadratic acceleration algorithm to search the 512-bit collision space of the `sha512` roots:

$$\text{Complexity}_{\text{Quantum}} = \sqrt{\mathbf{2^{256}}} = \mathbf{2^{128}} \text{ circuit operations}$$

Therefore, any double-signing attempt can be discovered publicly at a strict 128-bit post-quantum security floor. □

6.3 Unforgeability (EUF-CMA Resistance)

Theorem 3. *The OT-PQRS scheme is existentially unforgeable under adaptive chosen-message attacks (EUF-CMA) in the post-quantum setting, assuming the preimage and collision resistance of the* `sha512` *and* `sha256` *block primitives hold firmly up to a 128-bit complexity threshold.*

Proof. Suppose a post-quantum adversary $\mathcal{F}$ intends to construct a valid signature forgery $\mathbf{Sig}^* = (\mathbf{RT}^*, \mathbf{MTR}^*, \mathbf{SLR}^*, \mathbf{BDCP}^*)$ for a target public key $\mathbf{PK}^*$ on an arbitrary message $\mathbf{m}^*$ that was never explicitly signed by the legitimate credential holder.

1. **The Preimage Barrier of the Commitment Token:** The target public key is bound via $\mathbf{PK}^* = \mathbf{SKP} \oplus \mathbf{CMT}^*$. To forge a valid proof, $\mathcal{F}$ must supply a signature payload that reconstructs the precise commitment token:

$$\mathbf{CMT}^* = \texttt{sha512}(\mathbf{RT}^* \parallel \mathbf{MTR}^*)$$

If $\mathcal{F}$ attempts to craft a fraudulent tree folding path by inventing an arbitrary vector pair $\mathbf{SLR}^*$, they must force the parallel bottom-up hierarchical folding pass to resolve precisely to the fixed value $\mathbf{RT}^*$, while simultaneously forcing the `sha256` matrix mapping $\mathbf{HSLR}^*$ to align perfectly with $\mathbf{MTR}^*$.

2. **The Compression Extraction Reduction:** Every step up the tree architecture compresses a 1024-bit combined string down to a 512-bit parent node hash via the verifier's loop equation:

$$\text{ParentNode} = \texttt{sha512}\big((\text{Node}_{\mathbf{L}} \oplus \mathbf{Sig}.\mathbf{SLR}_{\mathbf{i},\text{left}}) \parallel (\text{Node}_{\mathbf{R}} \oplus \mathbf{Sig}.\mathbf{SLR}_{\mathbf{i},\text{right}})\big)$$

Because the message context is embedded inside the leaf boundary token $\mathbf{M}^* = \texttt{sha512}(\mathbf{m}^* \parallel \mathbf{I}^*) \oplus \mathbf{I}^*$, any mutation of the target message completely alters the final index leaf value. For the fraudulent path to resolve to $\mathbf{RT}^*$ over a mutated leaf node without breaking adjacent unassociated public keys, $\mathcal{F}$ must find a target preimage or an exact collision against the intermediate `sha512` compression rounds.

3. **Quantum Preimage Search Bound:** According to standard quantum cryptanalysis, evaluating a preimage over an ideal random oracle using Grover's algorithm provides a maximum quadratic speedup. For a 256-bit or 512-bit block primitive containing no structural linear dependencies or side-channel padding leakages:

$$\text{Complexity}_{\text{Quantum}} \geq \sqrt{\mathbf{2^{256}}} = \mathbf{2^{128}} \text{ quantum circuit operations}$$

Consequently, the primitive achieves existential unforgeability against chosen-message attacks, bound securely at **NIST Level 1 post-quantum thresholds**. □

7 Appendix: Architectural Flow and Structural Diagrams

This section provides formal topological, data-flow, and layerwise schematic diagrams mapping the structural interaction interfaces between the Partially Fixed Merkle Tree (**PFMT**) and the Batch Deniable Commitment Scheme (**BDCS**) across all system lifecycle epochs.

7.1 Hierarchical Key Isolation and Blinding Pipeline

The schematic diagram below maps the functional transformation layout where the underlying private credential parameters are synthesized into an unlinked, post-quantum secure 512-bit Public Key **PK** prior to ring assignment.

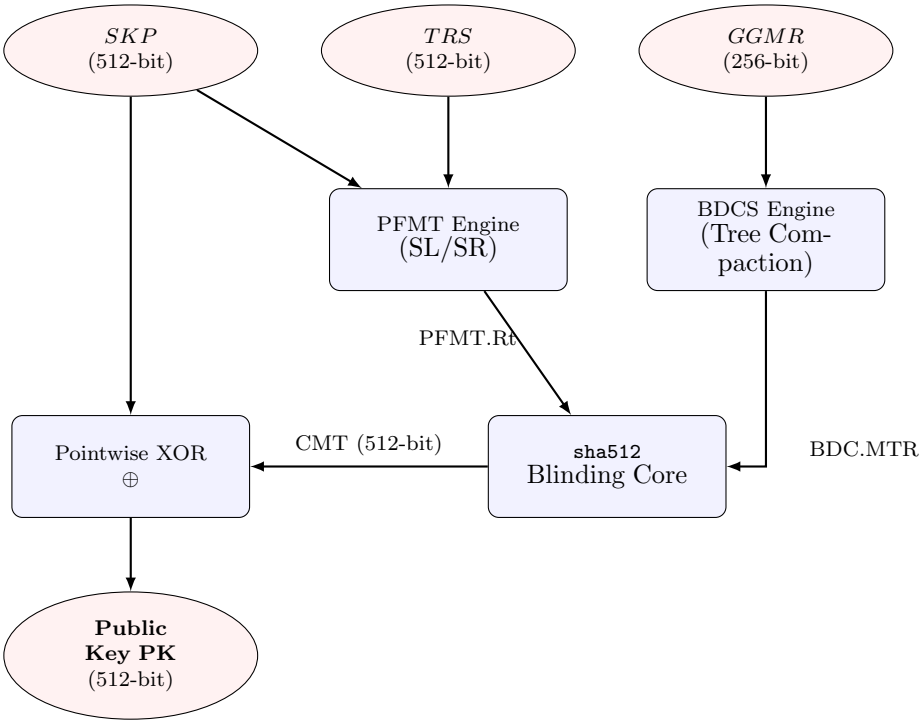


Figure 1: Asymmetric key isolation, commitment synthesis, and identity masking data-flow topology.

7.2 Phase-Split Key Generation Strategy

The diagram below details the exact processing footprint during the public key generation epoch. The framework isolates active path components from missing variables, mapping single-element targets into a dense commit structure while preserving companion registers as hidden blank states.

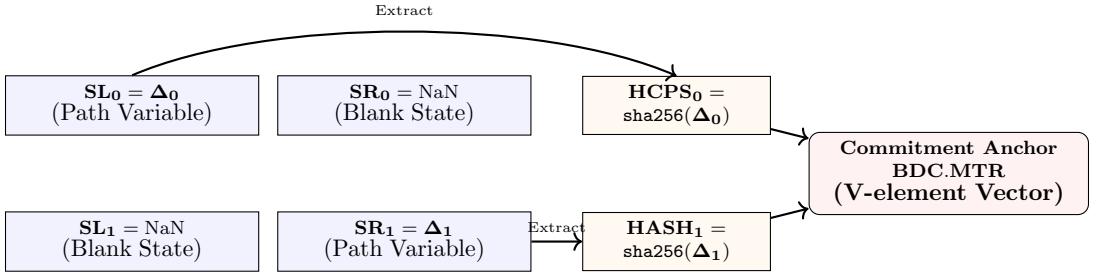


Figure 2: Phase-split path derivation and SL/SR vector extraction mapping during key generation.

7.3 The Signature Composition Loop

The block schematic below tracks the operational pipeline executing during the signature generation epoch. The runtime loop merges context structures, resolves unassigned state gaps, passes fields through parallel compression circuits, and aggregates parameters into a serialized wire transaction packet.

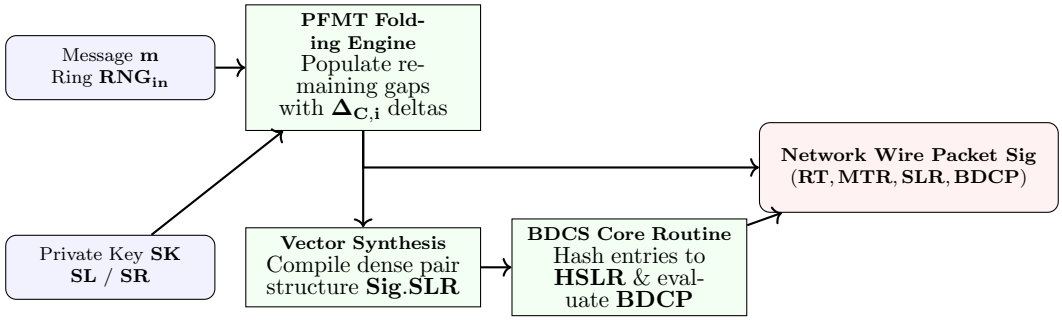


Figure 3: Signature generation pipeline execution matrix and runtime variable resolution flow.

7.4 PFMT Spatial Slicing and Layerwise Merkle Tree Folding

The topological chart below maps how the horizontal node vectors collapse bottom-up under a standalone public verification pass for an ad-hoc ring layout of height $\mathbf{V = 3}$ ($\mathbf{2^3 = 8}$ leaves).

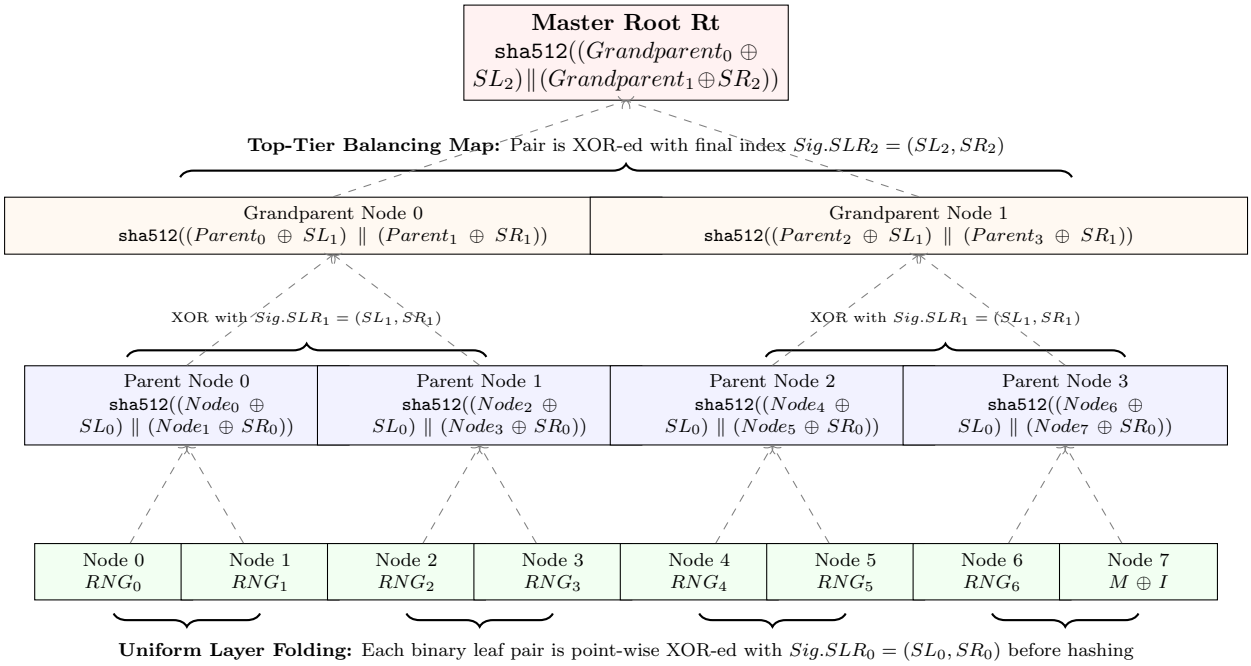


Figure 4: Levelwise uniform pair reordering and bracket-broadcast Merkle tree folding mappings for height $V = 3$.